

Projet pratique MLOps de A à Z

Construction d'un projet Machine Learning reproductible avec DVC, MLflow, Docker et GitHub Actions (CI)

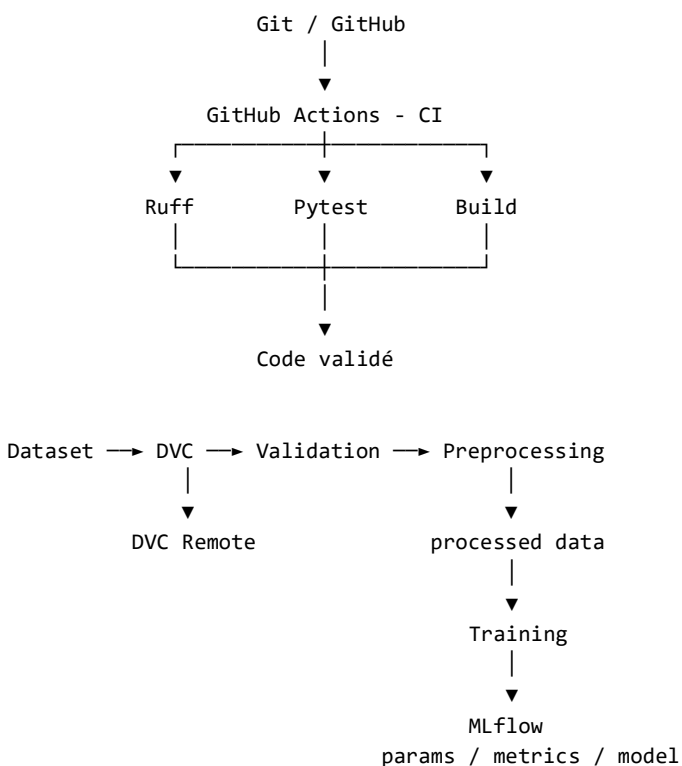
Objectif du projet

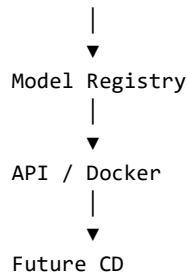
L'objectif est de construire un projet ML complet que l'on peut réellement pratiquer de bout en bout. Le cas choisi est une prédiction du prix d'un appartement à partir de caractéristiques comme la surface, le nombre de pièces et la localisation.

Le projet doit permettre de comprendre non seulement comment entraîner un modèle, mais surtout comment industrialiser son cycle de vie : préparer l'environnement, gérer les données, versionner les datasets avec DVC, reproduire le pipeline, suivre les expériences avec MLflow, tester le code avec pytest et automatiser les contrôles avec GitHub Actions.

- Entraîner un modèle de régression.
- Séparer code, données, modèle et configuration.
- Versionner le dataset avec DVC.
- Définir un pipeline DVC reproductible.
- Suivre les expériences avec MLflow.
- Tester le preprocessing et le code ML.
- Construire une CI GitHub Actions.
- Ajouter une API FastAPI et une image Docker en fin de parcours.
- Préparer le projet pour une future étape CD.

1. Architecture finale





GitHub Actions est utilisé ici principalement pour la CI. DVC et MLflow restent des outils spécialisés : GitHub Actions lance leurs commandes quand cela est nécessaire.

2. Ce que l'on va apprendre

À la fin, on doit être capable de répondre à ces questions :

- Comment organiser un projet ML proprement ?
- Comment un dataset est-il versionné sans mettre les gros fichiers dans Git ?
- Comment DVC sait-il qu'un stage doit être recalculé ?
- Comment reproduire le preprocessing et l'entraînement ?
- Comment enregistrer les paramètres et métriques d'une expérience ?
- Comment tester le code ML ?
- Comment GitHub Actions récupère le dépôt et exécute automatiquement les tests ?
- Que se passe-t-il lorsqu'un développeur ouvre une Pull Request ?
- Comment préparer le projet pour Docker et le futur déploiement ?

3. Prérequis

- Python 3.11 ou version compatible.
- Git et un compte GitHub.
- VS Code recommandé.
- DVC installé localement.
- Docker Desktop recommandé.
- Connaissances de base en pandas et scikit-learn.
- Notions de ligne de commande.

MLflow peut être utilisé d'abord localement. Pour un vrai travail d'équipe, on pourra ensuite déplacer le tracking vers un serveur MLflow.

4. Choix du problème ML

Nous allons traiter une régression : prédire le prix d'un appartement. Le dataset peut être réel ou synthétique au début. Le plus important pour l'exercice est d'avoir plusieurs variables numériques et catégorielles.

Exemple de colonnes :

```
surface      : float
rooms        : int
bedrooms     : int
city         : categorical
```

```
distance_km : float
price       : float # target
```

La colonne price est la cible. Les autres colonnes constituent les features.

5. Structure initiale du projet

```
mlops-house-price/
├── data/
│   ├── raw/
│   └── processed/
├── models/
├── src/
│   ├── __init__.py
│   ├── validate_data.py
│   ├── preprocess.py
│   ├── train.py
│   ├── evaluate.py
│   └── predict.py
├── tests/
│   ├── test_preprocess.py
│   ├── test_validation.py
│   └── test_train.py
├── notebooks/
├── configs/
│   └── config.yaml
├── .github/
│   └── workflows/
│       └── ci.yml
├── dvc.yaml
├── params.yaml
├── params.dvc
├── requirements.txt
├── pyproject.toml
├── Dockerfile
├── .gitignore
└── README.md
```

Cette séparation est importante : src contient le code, data contient les données, models les artefacts de modèle, tests les tests et .github/workflows les automatisations.

6. Préparer Git

```
git init
git add .
git commit -m "Initial project structure"
git branch -M main
git remote add origin <URL_DU_REPO>
git push -u origin main
```

Le code et les petits fichiers de métadonnées peuvent être versionnés par Git. Le dataset volumineux sera géré par DVC.

7. Préparer l'environnement Python

```
python -m venv .venv
```

```
# Windows  
.venv\Scripts\activate
```

```
pip install pandas numpy scikit-learn      pytest pytest-cov ruff dvc mlflow fastapi uvicorn pyyaml
```

Dans le projet final, les dépendances seront placées dans requirements.txt ou pyproject.toml afin que le runner GitHub Actions puisse recréer l'environnement.

8. Gestion du dataset avec DVC

Le principe est de ne pas faire de Git le stockage principal des gros datasets. DVC crée un petit fichier de métadonnées qui permet de versionner la donnée et de la retrouver dans un remote.

```
dvc init  
git add .dvc .gitignore  
git commit -m "Initialize DVC"
```

```
dvc add data/raw/houses.csv
```

```
git add data/raw/houses.csv.dvc  
git commit -m "Track raw dataset with DVC"
```

Après dvc add, le fichier houses.csv.dvc est versionné par Git, tandis que le contenu du dataset est géré par DVC.

9. DVC Remote

DVC a besoin d'un remote pour partager les données. Pour un exercice local, on peut utiliser un dossier distant local. Pour un projet réel, on peut utiliser un stockage compatible avec le projet : S3, Azure Blob, Google Cloud Storage, SSH, etc.

```
dvc remote add -d storage /chemin/vers/dvc-storage  
dvc push
```

Le principe devient alors : Git versionne les métadonnées et DVC remote conserve les données.

10. Préparer la validation des données

Avant de lancer l'entraînement, on veut éviter qu'un dataset invalide arrive dans le modèle.

```
# src/validate_data.py
```

```
import pandas as pd
```

```
REQUIRED_COLUMNS = [  
    "surface",  
    "rooms",  
    "bedrooms",
```

```

    "city",
    "distance_km",
    "price",
]

def validate(df):
    missing = [c for c in REQUIRED_COLUMNS if c not in df.columns]
    if missing:
        raise ValueError(f"Missing columns: {missing}")

    if df["surface"].isna().any():
        raise ValueError("surface contains missing values")

    if (df["surface"] <= 0).any():
        raise ValueError("surface must be positive")

    if (df["price"] <= 0).any():
        raise ValueError("price must be positive")

    if len(df) < 100:
        raise ValueError("Dataset is too small")

```

Cette validation peut ensuite être remplacée ou complétée par Pandera ou Great Expectations.

11. Preprocessing

Le preprocessing transforme les données brutes en données utilisables par le modèle. Pour les variables catégorielles, on peut utiliser OneHotEncoder ; pour les variables numériques, un SimpleImputer et éventuellement StandardScaler.

```

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_features = [
    "surface", "rooms", "bedrooms", "distance_km"
]

categorical_features = ["city"]

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features),
])

```

Le preprocessing doit être conservé avec le modèle final ou intégré dans une Pipeline scikit-learn afin d'appliquer exactement les mêmes transformations lors de l'inférence.

12. Entraînement

Nous allons commencer avec RandomForestRegressor. Le but n'est pas d'obtenir le meilleur modèle possible mais d'avoir une chaîne reproductible.

```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(
    n_estimators=200,
    max_depth=15,
    random_state=42,
)
```

Les paramètres comme n_estimators et max_depth seront idéalement centralisés dans params.yaml.

13. params.yaml

```
train:
  test_size: 0.2
  random_state: 42

model:
  n_estimators: 200
  max_depth: 15
  min_samples_split: 2
```

Cela permet de modifier les paramètres sans modifier directement le code et permet à DVC de détecter une modification de paramètre lorsqu'ils sont déclarés comme dépendances du stage.

14. Pipeline DVC

Nous allons définir plusieurs étapes : validation, preprocessing, puis entraînement. Une organisation possible est :

```
raw dataset
  ↓
validate
  ↓
preprocess
  ↓
processed dataset
  ↓
train
  ↓
model artifact
```

Exemple conceptuel de dvc.yaml :

```
stages:

  validate:
    cmd: python src/validate_data.py
    deps:
```

```

- data/raw/houses.csv
- src/validate_data.py

preprocess:
  cmd: python src/preprocess.py
  deps:
    - data/raw/houses.csv
    - src/preprocess.py
  outs:
    - data/processed/train.csv
    - data/processed/test.csv

train:
  cmd: python src/train.py
  deps:
    - data/processed/train.csv
    - data/processed/test.csv
    - src/train.py
    - params.yaml
  params:
    - model
    - train
  outs:
    - models/model.joblib

```

deps signifie dependencies. outs signifie outputs. params déclare les paramètres qui influencent le stage.

15. Comprendre dvc repro

DVC calcule les dépendances et détecte les changements. Si le fichier raw change, le preprocessing peut être relancé et le training peut devenir obsolète. Si uniquement train.py change, DVC n'a pas besoin de refaire le preprocessing si son output reste valide.

```
dvc repro
```

C'est l'un des points centraux de la reproductibilité MLOps : on ne veut pas exécuter manuellement dix commandes dans un ordre précis à chaque changement.

16. MLflow : tracking des expériences

Le script train.py va enregistrer les paramètres et métriques dans MLflow.

```

import mlflow
import mlflow.sklearn

with mlflow.start_run():
    mlflow.log_param("n_estimators", n_estimators)
    mlflow.log_param("max_depth", max_depth)

    model.fit(X_train, y_train)

    predictions = model.predict(X_test)
    rmse = mean_squared_error(
        y_test, predictions, squared=False
    )
    r2 = r2_score(y_test, predictions)

```

```

mlflow.log_metric("rmse", rmse)
mlflow.log_metric("r2", r2)

mlflow.sklearn.log_model(
    model,
    "model"
)

```

On peut ensuite comparer plusieurs runs : différents paramètres, différents datasets ou différentes versions de code.

17. Lancer MLflow localement

```
mlflow ui
```

L'interface permet de consulter les runs, paramètres, métriques et artefacts. Dans un projet collaboratif, un serveur de tracking distant sera préférable.

18. Tests unitaires

Avant de mettre le projet dans CI, il faut écrire des tests.

```

# tests/test_validation.py

import pandas as pd
import pytest
from src.validate_data import validate

def test_valid_data():
    df = pd.DataFrame({
        "surface": [80, 100],
        "rooms": [3, 4],
        "bedrooms": [2, 3],
        "city": ["Fes", "Rabat"],
        "distance_km": [2.0, 5.0],
        "price": [800000, 1200000],
    })

    validate(df)

def test_negative_surface():
    df = pd.DataFrame({
        "surface": [-10],
        "rooms": [3],
        "bedrooms": [2],
        "city": ["Fes"],
        "distance_km": [2.0],
        "price": [500000],
    })

    with pytest.raises(ValueError):
        validate(df)

```

19. Tests du preprocessing

```
def test_preprocessor():
    X = sample_dataframe()

    transformed = preprocessor.fit_transform(X)

    assert transformed.shape[0] == len(X)
    assert transformed.shape[1] > 0
```

On peut également tester que des catégories inconnues sont acceptées si OneHotEncoder est configuré avec `handle_unknown='ignore'`.

20. Coverage

```
pytest --cov=src --cov-report=term-missing
```

On peut ajouter une règle minimale, par exemple 80 %, mais le seuil doit rester adapté au projet. Une couverture élevée ne garantit pas à elle seule la qualité des tests.

21. Ruff

```
ruff check .
ruff format --check .
```

Ruff check vérifie les règles de lint. Ruff format --check vérifie que le code respecte le format attendu sans modifier les fichiers.

22. pyproject.toml

```
[project]
name = "mlops-house-price"
version = "0.1.0"
requires-python = ">=3.11"

[tool.pytest.ini_options]
testpaths = ["tests"]

[tool.ruff]
line-length = 88

[tool.ruff.lint]
select = ["E", "F"]
```

Dans un vrai projet, les dépendances peuvent aussi être déclarées dans la section project ou via un outil de gestion de dépendances.

23. GitHub Actions : première CI

La première CI doit rester simple : récupérer le code, installer Python et les dépendances, lancer lint et tests.

```
# .github/workflows/ci.yml

name: CI
```

```

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  quality:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest pytest-cov ruff

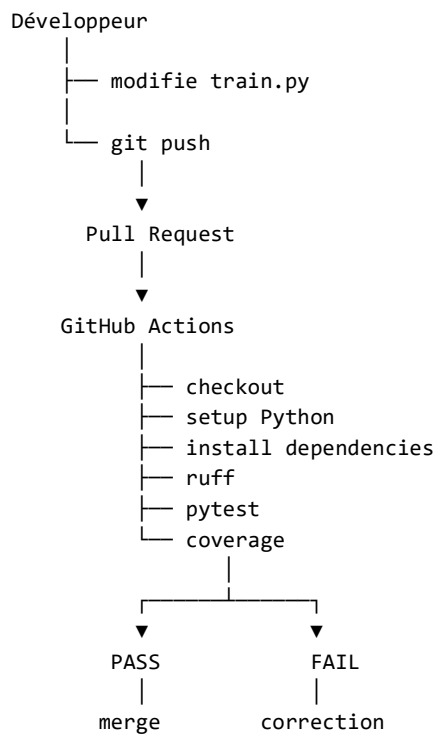
      - name: Lint
        run: ruff check .

      - name: Tests
        run: pytest --cov=src --cov-fail-under=80

```

À chaque Pull Request vers main, GitHub Actions démarre un runner et exécute ces étapes.

24. Ce qui se passe réellement lors d'une Pull Request



La CI ne décide pas si le modèle est scientifiquement le meilleur. Elle vérifie surtout que le changement de code respecte les contrôles automatisés définis.

25. Ajouter DVC à la CI : nuance importante

Pour une CI de Pull Request, il ne faut pas forcément télécharger un énorme dataset ou exécuter un entraînement complet. On peut garder la CI rapide.

Deux approches sont possibles :

- CI légère : lint + unit tests + tests d'intégration utilisant de petites données de test.
- Pipeline de training séparé : dvc pull + validation + dvc repro + entraînement + MLflow.

Cette séparation est particulièrement importante lorsque l'entraînement prend plusieurs minutes ou heures.

26. Workflow GitHub Actions pour training

```
name: ML Training
```

```
on:
```

```
  workflow_dispatch:
```

```
jobs:
```

```
  train:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

- name: Checkout
 uses: actions/checkout@v4
- name: Setup Python
 uses: actions/setup-python@v5
 with:
 python-version: "3.11"
- name: Install dependencies
 run: |
 pip install -r requirements.txt
 pip install dvc
- name: Pull dataset
 run: dvc pull
- name: Validate data
 run: python src/validate_data.py
- name: Reproduce pipeline
 run: dvc repro
- name: Run tests
 run: pytest

Pour un serveur MLflow distant, les variables de connexion seraient fournies par des secrets GitHub.

27. Secrets GitHub

```
- name: Train
  env:
    MLFLOW_TRACKING_URI: ${ secrets.MLFLOW_TRACKING_URI }
    MLFLOW_TRACKING_USERNAME: ${ secrets.MLFLOW_TRACKING_USERNAME }
    MLFLOW_TRACKING_PASSWORD: ${ secrets.MLFLOW_TRACKING_PASSWORD }
  run: python src/train.py
```

Ne jamais écrire les credentials directement dans le repository.

28. Gestion du modèle

Le modèle produit par train.py doit être un artefact clairement identifiable. Par exemple models/model.joblib. DVC peut versionner cet output si le modèle est suffisamment important pour être géré comme artefact de pipeline. MLflow peut également stocker le modèle et ses métadonnées.

Il faut éviter de mélanger les responsabilités : DVC peut gérer la reproductibilité des artefacts et datasets, tandis que MLflow est particulièrement utile pour le tracking et le cycle de vie des modèles.

29. Model Registry

Dans une architecture plus avancée, un modèle entraîné passe par plusieurs états conceptuels : entraîné, évalué, candidat, staging, production. Le Model Registry aide à gérer les versions et la promotion selon les règles de l'organisation.

```
Training
↓
MLflow Run
↓
Evaluation
↓
Registered Model
↓
Candidate
↓
Staging
↓
Production
```

La promotion ne doit pas être basée uniquement sur une métrique isolée. Les critères dépendent du projet.

30. Dockerisation de l'API

Après avoir stabilisé le pipeline ML, on peut exposer le modèle via FastAPI.

```
# Dockerfile

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```

COPY src/ ./src/
COPY models/ ./models/

CMD ["uvicorn", "src.main:app",
     "--host", "0.0.0.0",
     "--port", "8000"]

docker build -t mlops-house-price:1.0 .
docker run -p 8000:8000 mlops-house-price:1.0

```

31. Test d'intégration de l'API

```

from fastapi.testclient import TestClient
from src.main import app

client = TestClient(app)

def test_predict():
    response = client.post(
        "/predict",
        json={
            "surface": 80,
            "rooms": 3,
            "bedrooms": 2,
            "city": "Fes",
            "distance_km": 3.0
        }
    )

    assert response.status_code == 200
    assert "prediction" in response.json()

```

Ce test vérifie plusieurs composants ensemble : validation de la requête, chargement du modèle, preprocessing et réponse HTTP.

32. Ajouter le build Docker à la CI

```

docker-build:
  needs: quality
  runs-on: ubuntu-latest

  steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Build Docker image
      run: docker build -t mlops-house-price:test .

```

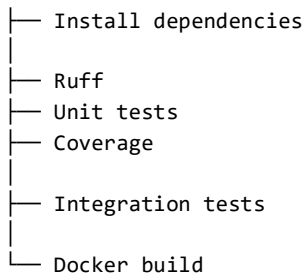
Pour une CI, construire l'image est utile pour détecter les erreurs de packaging. Le push vers un registry peut être réservé à la branche main ou à une release selon la stratégie choisie.

33. Workflow CI final recommandé

```

CI
|
├─ Checkout
├─ Setup Python

```



L'objectif est de garder ce workflow suffisamment rapide pour être exécuté souvent.

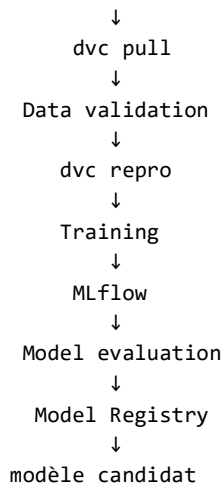
34. Ce qui ne doit pas forcément être dans la CI

- Entraînement complet d'un gros modèle à chaque Pull Request.
- Téléchargement d'un dataset de plusieurs dizaines de gigaoctets pour chaque changement de code.
- Déploiement automatique en production à chaque commit sans règles de protection.
- Expérimentations longues et coûteuses.

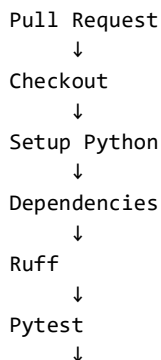
Ces opérations peuvent être déplacées dans des workflows spécialisés.

35. Workflow training complet

Nouveau dataset / déclenchement manuel



36. Workflow CI complet



Coverage
↓
Integration tests
↓
Docker build
↓
CI PASS

37. Ordre recommandé pour pratiquer

Ne cherche pas à construire tout le système dès le premier jour. Construis-le par étapes et vérifie chaque étape.

1. Créer le projet Python et entraîner un premier modèle.
2. Séparer preprocessing, training et evaluation.
3. Ajouter les tests unitaires.
4. Ajouter Ruff.
5. Ajouter pytest-cov.
6. Initialiser DVC et versionner le dataset.
7. Créer le pipeline dvc.yaml.
8. Modifier les données et observer dvc repro.
9. Ajouter MLflow et comparer plusieurs runs.
10. Créer la CI GitHub Actions.
11. Ajouter les tests d'intégration.
12. Ajouter Docker.
13. Construire l'image dans GitHub Actions.
14. Ajouter ensuite un workflow training séparé.
15. Enfin, étudier le CD et le déploiement.

38. Exercices pratiques

Exercice 1 — Reproductibilité

Modifie une valeur dans le dataset puis exécute dvc repro. Observe quels stages sont recalculés.

Exercice 2 — Paramètres

Change max_depth dans params.yaml puis exécute dvc repro. Vérifie que le training est relancé.

Exercice 3 — Bug

Introduis volontairement une erreur dans preprocessing.py et vérifie que pytest ou la CI la détecte.

Exercice 4 — CI

Crée une Pull Request contenant une erreur de lint. Observe le résultat GitHub Actions.

Exercice 5 — MLflow

Entraîne trois modèles avec des paramètres différents et compare leurs RMSE/R².

Exercice 6 — Data validation

Ajoute une valeur négative de surface dans le dataset et vérifie que le pipeline s'arrête avant l'entraînement.

Exercice 7 — Docker

Construis l'image et lance l'API localement.

Exercice 8 — Integration test

Écris un test qui appelle /predict et vérifie la structure de la réponse.

39. Arborescence finale

```
mlops-house-price/
├── .github/
│   ├── workflows/
│   │   ├── ci.yml
│   │   └── train.yml
│   └──
├── data/
│   ├── raw/
│   │   └── houses.csv.dvc
│   └── processed/
├── models/
│   └── model.joblib
├── src/
│   ├── validate_data.py
│   ├── preprocess.py
│   ├── train.py
│   ├── evaluate.py
│   └── main.py
├── tests/
│   ├── test_validation.py
│   ├── test_preprocess.py
│   └── test_api.py
├── dvc.yaml
├── params.yaml
├── pyproject.toml
├── requirements.txt
├── Dockerfile
├── .gitignore
└── README.md
```

40. Checklist finale

- Le projet fonctionne localement.
- Le dataset est versionné avec DVC.
- Un DVC remote est configuré pour le partage.
- Le pipeline DVC est reproductible avec `dvc repro`.
- Les paramètres sont séparés du code.
- Les données sont validées avant l'entraînement.
- Le preprocessing est testé.
- Le training est testé autant que possible.
- Les expériences sont enregistrées dans MLflow.

- Les métriques permettent de comparer les runs.
- Le modèle est sauvegardé comme artefact.
- Ruff est exécuté automatiquement.
- Pytest est exécuté automatiquement.
- Coverage est contrôlée.
- Les tests d'intégration sont présents.
- GitHub Actions exécute la CI sur Pull Request.
- Le build Docker est vérifié.
- Les secrets ne sont pas commités.
- Le training lourd est séparé de la CI rapide.
- Le projet est prêt à recevoir une future étape CD.

41. Résumé : qui fait quoi ?

Git/GitHub

→ version du code et collaboration

DVC

→ version des datasets + pipeline + artefacts

Data Validation

→ vérification de la qualité des données

MLflow

→ expériences + paramètres + métriques + modèles

Pytest

→ tests

Ruff

→ qualité statique du code

GitHub Actions

→ automatisation / orchestration de la CI

Docker

→ packaging de l'application

Registry

→ stockage des images

Model Registry

→ gestion des versions de modèles

CD

→ livraison et déploiement

42. Résultat attendu du projet

Le résultat final n'est pas simplement un fichier model.pkl. Le véritable livrable est un système reproductible dans lequel une autre personne peut récupérer le repository, récupérer la bonne version des données, reconstruire le pipeline, comprendre comment le modèle a été entraîné, vérifier sa qualité et exécuter automatiquement les contrôles CI.

C'est précisément cette différence entre « entraîner un modèle » et « construire un système ML reproductible » qui constitue le cœur de la pratique MLOps.